

Incapsulamento

Incapsulamento

Come usare `public` e `private` per proteggere i dati nelle classi C++.

Cos'è l'incapsulamento?

Immagina un distributore automatico. Puoi premere i pulsanti (interfaccia pubblica), ma non puoi mettere le mani nel meccanismo interno (dati privati). Questo è l'**incapsulamento**: nascondere i dettagli interni e offrire solo un accesso controllato dall'esterno.

In pratica: i dati di una classe vengono dichiarati **privati**, così nessuno può modificarli direttamente dall'esterno. Per accedervi, si usano metodi pubblici appositamente creati.

Il problema senza incapsulamento

Quando i dati sono pubblici, chiunque può modificarli — anche con valori assurdi:

```
class Eta {
public:
    int valore;    // pubblico: chiunque può modificarlo
};

int main() {
    Eta e;
    e.valore = -5;    // nessuno impedisce valori assurdi!
    e.valore = 9999;  // nessun controllo!
    return 0;
}
```

La soluzione: dati privati, metodi pubblici

```
class Eta {
private:
    int valore;    // privato: nessuno può accedervi direttamente
                  // dall'esterno

public:
    // Setter: imposta il valore, ma solo se è valido
    void setValore(int v) {
        if (v >= 0 && v <= 120) {
            valore = v;
        } else {
            cout << "Eta non valida!" << endl;
        }
    }

    // Getter: restituisce il valore corrente
    int getValore() const {
        return valore;
    }
};

int main() {
    Eta e;
    e.setValore(16);    // OK
    e.setValore(-5);    // "Eta non valida!" – il controllo blocca il
    valore
    e.setValore(9999);  // "Eta non valida!"

    cout << e.getValore() << endl;    // 16

    // e.valore = 5;    // ERRORE di compilazione: valore è privato!
    return 0;
}
```

I tre livelli di accesso

In C++ ci sono tre parole chiave per controllare chi può accedere ai dati:

Parola chiave	Chi può accedere
<code>public</code>	Tutti — dall'interno della classe e dall'esterno
<code>private</code>	Solo dall'interno della classe
<code>protected</code>	Dalla classe e dalle classi che la estendono (ereditarietà)

Esempio completo: classe Persona

```
#include <iostream>
#include <string>
using namespace std;

class Persona {
private:
    string nome;    // dati privati
    int eta;
    string email;

public:
    // Costruttore: usa i setter per validare i valori fin dall'inizio
    Persona(string n, int e, string em) {
        nome = n;
        setEta(e);    // passa attraverso il setter
        email = em;
    }

    // Getter: leggono i dati privati (sola lettura)
    string getName() const { return nome; }
    int getEta() const { return eta; }
    string getEmail() const { return email; }

    // Setter con validazione
    void setEta(int e) {
        if (e >= 0 && e <= 150) {
            eta = e;
        } else {
            cout << "Eta non valida!" << endl;
            eta = 0;
        }
    }

    void setEmail(string em) {
        if (em.find('@') != string::npos) {    // deve contenere @
            email = em;
        } else {
            cout << "Email non valida!" << endl;
        }
    }

    // Metodo per stampare le informazioni
    void stampa() const {
        cout << "Nome: " << nome << endl;
    }
}
```

```

        cout << "Eta: " << eta << endl;
        cout << "Email: " << email << endl;
    }
};

int main() {
    Persona p("Alice", 16, "alice@esempio.it");
    p.stampa();

    p.setEta(17); // OK
    p.setEmail("non-una-email"); // Email non valida!
    p.setEmail("alice.nuovo@esempio.it"); // OK

    cout << "Nuova eta: " << p.getEta() << endl;
    cout << "Nuova email: " << p.getEmail() << endl;

    return 0;
}

```

Il vantaggio nascosto: libertà di cambiare l'interno

L'incapsulamento ti permette di modificare come funziona internamente una classe **senza dover cambiare il codice che la usa**:

```

// Versione 1: memorizza la temperatura in Celsius
class Termometro {
private:
    double celsius;
public:
    void setTemperatura(double t) { celsius = t; }
    double getTemperatura() const { return celsius; }
};

// Versione 2: internamente usa Kelvin, ma l'interfaccia è identica!
// Chi usa questa classe non sa (e non deve sapere) del cambiamento
// interno.
class Termometro {
private:
    double kelvin; // cambiato internamente
public:
    void setTemperatura(double celsius) { kelvin = celsius + 273.15; }
    double getTemperatura() const { return kelvin - 273.15; }
};

```

Chi scrive `t.setTemperatura(100)` non deve cambiare nulla — l'interfaccia è la stessa.

Ricorda: nelle `class`, tutto è privato per default

```
class Esempio {  
    int x;    // privato per default (nessun public/private specificato)  
  
    public:  
        int y;    // pubblico (esplicitamente)  
};
```

È buona abitudine specificare sempre `public:` e `private:` per chiarezza, anche se tecnicamente non sarebbe necessario.